Debugging Strategies

Learn to understand, design, build, test and explain AI systems responsibly.

Project: Transparent Book Recommender

TEACHER GUIDE · 1 OF 9

Debugging Strategies

> Prepare Debugging Strategies: check the target, prerequisites, resources and adaptable teaching schedule.

Teach the learning cycle

Primary target: Design revealing tests for a function

Session · Sequence and minutes · Resources

> **Session:** Session 1 · Understand · 45 min
>
> **Sequence and minutes:** 0–7: recall and prerequisite; 7–15: inspect the concrete case and predict; 15–30: teacher models the worked trace; 30–40: learners explain a step using the text and visual; 40–45: check the core idea.
>
> **Resources:** Use Your mission, Notice the evidence, See how it works, Compare the cases, Words that unlock the idea and Follow a worked example; record the brief recall in the workbook.

> **Session:** Session 2 · Practise · 45 min
>
> **Sequence and minutes:** 0–5: retrieve the procedure; 5–15: W2 completion practice; 15–23: explain and check with a partner; 23–30: each learner answers the misconception check; 30–40: targeted reteaching or a harder variation; 40–45: prepare the investigation.

> **Resources:** Use Find and repair a mistake, Practise together, Run the investigation and Build your understanding; use the matching workbook W2 and misconception check.

> **Session:** Session 3 · Investigate and transfer · 45 min
>
> **Sequence and minutes:** 0–5: retrieve the decision rule; 5–20: investigate or trace; 20–30: start the independent case; 30–38: connect the project; 38–43: act on feedback; 43–45: plan the return check.
>
> **Resources:** Use Run the investigation, Connect your project, Show what you know, Study a complete case and Try a new case independently. Allow extra time for the full task set. Use a coding lab only when this week actually provides one; Your lesson route is the planning section.

This is a starting allocation of 135 minutes, not a validated duration. Preserve all named topics. Schedule extra time for connected examples, the full investigation, independent cases and project building.

Repeated copies of a case share one recorded attempt; they are not new **evidence**.

Case exposure check: this source example appears in ai-2/6, python-bridge/12. Reuse a familiar case for recall or modelling; verify that the case used to assess transfer has not already received feedback.

Retrieval answer guidance

Loops and Stopping

Without opening the old lesson, explain one key idea from Loops and Stopping. Give an example and a mistake that the idea helps you avoid.

Look for: Trace variable values before and after each loop iteration

Reference example, not the only acceptable wording: 2, 3 and 4; the stop value is excluded. The body runs three times.

Branching Logic

Without opening the old lesson, explain one key idea from Branching Logic. Give an example and a mistake that the idea helps you avoid.

Look for: Trace the selected branch in an if elif else chain

Reference example, not the only acceptable wording: if tests negative first; elif checks <= 12 only after that fails; else handles the rest.

Unindented print runs after any selected branch, whose assignment gives label a value.

Use the student module and workbook section with this exact week title. The worked case, independent question and answers below form one aligned lesson sequence.

What the learner must demonstrate

Intended score is 3 per matching trait. Bug: score starts at 3 and adds 3 for each match. Give expected and actual results for 0,1,2 matches and repair the cause.

Concept to explain

Good **debug**ging isolates a cause with **controlled test**s.

Term	Meaning
bug	A defect that makes a program behave differently from its intended specification.
debug	Find and repair the cause of a program's incorrect behaviour.
controlled test	A test with specified inputs, conditions and expected measurements.
retest	Run a test again after a change or under specified new conditions.

Prepare the class

Use the supplied scenario, student illustrations and blank paper. For numeric work, offer counters or a calculator after learners show the setup. Fictional records are supplied; personal data and paid accounts are unnecessary.

Alternative compact allocation

> This compact option is not the full-week schedule. Use the main teaching cycle and allow extra sessions for the investigation, independent work and project.

> Allow 80 minutes: recall 8; explanation 12; worked demonstration 15; guided practice 20; independent assessment 15; project and reflection 10. Extend the lab into a second session when needed.

Preparation for the connected explanation

Teach Week 12 as a two-session investigation using fictional data. A function can pass one example by coincidence.

Start with a **written rule**, calculate expected results independently, and include ordinary, zero and **boundary** cases. We will test a subtraction function under the contract 0 <= used <= capacity.

Assess the three stated outcomes through the independent task, not by copying the worked result.

Readiness answer

7.

Python 3 editor with a fresh local practice file

Paper trace table and the student workbook

Fictional **input** records from this lesson; no real student identifiers

End goal for the pictorial case

By the end, I can test a function with expected values that distinguish a particular bug.

This added worked case illustrates the week's main idea. Retain all original and connected topics; schedule its practice within the teaching cycle.

TEACHER GUIDE · 2 OF 9

Explain the mechanism

> Model Debugging Strategies using the supplied case, labelled mechanism and reasoning. Make the scope of each conclusion explicit.

Input → Process → Output

Trace the information. At each stage, ask what changed and how you could test it.

Which stage could fail even when the other two are correct? Explain:

Predict first. Then check each step. A correct answer without visible reasoning is incomplete **evidence**.

1. Observed: wet-weather case recommends sandals.

2. Divide: **input** correct? yes. rain branch reached? no.

3. Cause: condition says rain = false instead of true.

4. Fix that condition only; rerun the original case and a dry-weather case.

Recalculate, retrace, or restate the reasoning in your own representation:

CHECK Did the **evidence** answer the exact question? Did the total, path, or source record stay consistent?

Concrete teaching case

The teaching case

Award two points per matching trait. A bug awards two for every trait.

The teaching case

Test one match and one nonmatch.

Exact explanation to model

Why the result follows

Expected 2; buggy result 4. Put the addition inside the match condition. Test zero, one and two matches for 0,2,4.

Explain the added worked example

A function can pass one example by coincidence. Start with a **written rule**, calculate expected results independently, and include ordinary, zero and **boundary** cases.

We will test a subtraction function under the contract 0 <= used <= capacity.

Explain the added worked example · 1

Each parenthesized group is a tuple, an ordered group of values. The loop unpacks each group into three names. Equality compares the actual result to the independently calculated expected result.

The worked script prints True three times. An assert actual == expected could instead stop at the first mismatch with **Assertion**Error.

Key terms

Term	Meaning
Test case	Inputs and an expected result
Boundary	An edge of a permitted range
Regression	A previously working case that fails after a change
Assertion	A check that raises an error when its condition is false

Model the reasoning aloud

Use the complete worked example below before asking learners to complete W2. Reveal a small part, invite a **prediction**, then explain the deciding step.

These teacher notes contain solutions; do not reveal the independent case answer during modelling.

> **Would replacing the body with return 5 pass every worked case? Identify the first revealing test.**

Thinking move	Teacher prompt
Plan	What is given? What is the question? Which procedure fits these facts?
Do	Point to each input and intermediate step in the worked trace. Name the rule that allows the next step.
Check	Does the output answer the question? Which assumption or limitation prevents a larger claim?

Use the diagram and verbal explanation together. Ask learners to match a sentence to a specific visual step; do not assign learners fixed visual or auditory learning styles.

A pictorial worked case: Debugging Strategies

A fictional shelf function should count books in complete bundles plus loose books.

The facts we will use

> **Supplied facts**
>
> Correct definition: shelf(bundle_count,loose) returns bundle_count*6+loose.

Supplied facts

Faulty copied body returns (bundle_count+loose)*6. Predict tests before running.

AI ACADEMY / AI-2/6

Expected outputs versus a specific bug

	Bundles	Loose	Correct	Faulty
Test 1	2	1	13	18
Test 2	0	3	3	18
Test 3	2	0	12	12

Read a cell using both its row and column labels.

Follow the labelled picture with the matching explanation. Open the vector diagram

AI ACADEMY / AI-2/6

Debugging Strategies

Start with the supplied facts

Follow each arrow. Ask which fact allows the next step.



1. Expected normal test

For (2,1), correct result is 2 x 6+1=13.



2. Faulty trace

Copied code gives (2+1) x 6=18.



3. Zero-bundle test

For (0,3), correct 3; copied code 18.



4. Nonrevealing test

For (2,0), both return 12; a pass does not locate every bug.

CHANGE ONE THING

Repair the body to bundle_count*6+loose.

Explain the changed result using the matching lesson.

Follow the labelled picture with the matching explanation. Open the vector diagram

Read the picture one step at a time

STEP	WHAT HAPPENS AND WHY
1. Expected normal test	For (2,1), correct result is 2×6+1=13.
2. Faulty trace	Copied code gives (2+1)×6=18.
3. Zero-bundle test	For (0,3), correct 3; copied code 18.
4. Nonrevealing test	For (2,0), both return 12; a pass does not locate every bug.

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

Why this result follows

Why the result follows

Test cases need not all expose the same error. Here loose books must be added after multiplying bundles. A zero-loose case cannot distinguish the two formulas because multiplying zero loose books changes nothing.

Keep expected and actual columns so an observed **output** does not quietly become its own expected answer.

What this example does not establish

Keep the stated limit

These three tests check a specified formula and bug; they do not prove correct handling of every type or invalid request.

Change one thing in the pictured case

Change one thing

Change: Repair the body to bundle_count*6+loose.

Predicted result

Prediction: Expected results for (2,1),(0,3),(2,0) are 13,3,12.

Why the result follows

Reason: Only bundles are groups of six; loose books are counted individually.

Ask while showing the picture

Ask while showing the picture · 2

Cover the result. Ask learners to name the **input**s and predict the next step. Uncover one step, connect the icon to the actual procedure, and explain its **evidence**. Then change the supplied condition and compare.

The picture is a representation; it does not document execution of a real AI model.

Demonstrate and guide practice

> Demonstrate Debugging Strategies, reduce support deliberately and check individual reasoning before discussing answers.

Board sequence

1. Put the facts on the board

Write the given facts: Award two points per matching trait. A bug awards two for every trait.

2. State the decision to explain

Write the question to explain: Test one match and one nonmatch.

3. Reveal the worked reasoning

Reveal after **prediction**s: Expected 2; buggy result 4. Put the addition inside the match condition. Test zero, one and two matches for 0,2,4.

Hold back the independent task

Keep for an independent attempt: A loop should run four times but runs five. You change its colour and stopping condition at once. Which single change is relevant to test first?

Classroom dialogue

Ask the class

Ask: Which facts are given, and which would we have to assume? Learners mark relevant details before discussing answers.

Language to model

Say: Predict first. Show the rule, information path or calculation that would produce the result. A correct guess is not yet an explanation.

Reveal reasoning one step at a time. Explain why a tempting alternative fails. Write units and denominators for arithmetic, trace conditions for decisions, and identify supporting sources for claims.

In pairs, one learner solves while the other checks **evidence**; exchange roles. Remove the worked answer before independent assessment.

Model these guided questions

It passes (8, 3) but fails (8, 8), returning 5 instead of 0. Exit: No. Negative counts are outside our nonnegative-**input** contract and were not tested; rejection behavior must first be defined.

Case-specific teaching sequence

Stage	Teacher action
Session 1 start to 7 minutes	Retrieve the prerequisite and explain the purpose in the opening section.
7 to 20 minutes	Read the worked example and diagram; ask learners to trace each step aloud.
20 to 33 minutes	Complete W1 and W2, then compare predictions with observed results.
33 to 45 minutes	Collect the guided prediction before running; discuss a mismatch and its cause.
Session 2 start to 8 minutes	Retrieve the main rule without showing the previous answer.
8 to 20 minutes	Use W3 to diagnose and repair the stated failure; rerun an earlier case.
20 to 35 minutes	Collect W4 independently, including code or plan, predictions and explanations, before hints.
35 to 45 minutes	Discuss W5 and the exit; use the fresh retry when support was needed.

Reduce help deliberately

Completion practice

Completion task: A faulty function always returns 4. Which of your three cases pass and fail?

Reasoning to listen for

Teacher check: Only (6, 2) passes. The full and unused cases fail.

Start by identifying the given **input**s together. Leave the intermediate steps blank. If needed, model one step, then withdraw that help on the next step. Move to a full trace only when the partial trace is understood.

For partners, alternate explainer and checker; collect a response from every learner before discussion.

Individual reasoning check

Use the worked case for Testing and **debug**ging functions. Proposed explanation: “A successful run proves the conclusion”. Decide whether this explanation is supported.

Point to the step or supplied fact that decides; explain your repair if needed.

Expected repair

Expected correction: Passing this small suite supports these particular cases. It neither proves every allowed **input** correct nor checks invalid **input** handling.

A test outside the contract needs a separately specified expected behavior.

Collect explanations, not only yes/no votes. If a misunderstanding is common, re-model the relevant step for the class.

If it affects a few learners, give a short targeted group explanation while others solve a more demanding variation.

Pictorial practice question

> Which test most directly isolates the handling of loose books, and what should it produce?

Reasoned teacher answer

(0,3) isolates loose books: expected 3. The faulty body gives 18.

Changed-case reasoning: Expected results for (2,1),(0,3),(2,0) are 13,3,12. Only bundles are groups of six; loose books are counted individually.

Collect each learner's explanation before discussion. Use the first missing connection between **input**, deciding step and result to choose support. This question is worked-case practice, not fresh mastery **evidence**.

Run the weekly evidence lab

> Run the Debugging Strategies investigation and use the first unsupported step to choose a teaching response.

Words You Can Use to Reason

A definition matters when you can recognize the idea, use it, and contrast it with a near-miss.

Predict → Test → Record → Explain

Keep expected and actual results separate. Surprises are useful data when recorded honestly.

Use normal, **boundary**, and unexpected test cards. Mark the first step where expected and actual behavior differ; change one cause; rerun all three.

> What did the hardest test teach you that the normal test did not?

Make the lab runnable

Use the concrete case as the shared normal case and the independent question as a second case. Each pair creates a **boundary** or missing-information case and predicts its result before testing.

Keep the expected result separate from an observed mistake.

Record	What to capture
Input or evidence	Exact data, instruction or claim
Expected result	Answer or safe fallback declared before the run
Observed result	Actual result, including errors
Explanation	Rule, calculation or evidence explaining the outcome
Next test	One justified change and a new prediction

Diagnose and reteach

For guessing, return to given facts and request a trace. For vocabulary without application, use one example and one non-example. For arithmetic errors, check the setup before the calculation.

Finish with a different uncoached case before recording mastery.

Address these additional misconceptions

A successful run proves the conclusion: Passing this small suite supports these particular cases. It neither proves every allowed **input** correct nor checks invalid **input** handling.

A test outside the contract needs a separately specified expected behavior.

Matching the example is enough: Require the new W4 case, explicit **prediction**s and an explanation. Fresh retry: bundles * 3 + singles with (3, 1), (0, 1), (3, 0) expects 10, 1, 9. A constant return 10 passes only the first.

A convenient shortcut preserves meaning: A faulty return capacity + used gives 11, 16 and 8. The unused case passes even though the rule is wrong. Repair the operator to subtraction and rerun all three cases.

Never change an expected result merely to match faulty **output**.

Match the support to the first error

What the response shows	Next teaching action
Missing or misread facts	Read the case aloud, mark supplied versus unknown facts, and let the learner restate it.
Wrong procedure	Return to the deciding step in the worked case. Contrast it with the proposed incorrect explanation.
Arithmetic or implementation error	Trace intermediate values or lines; preserve the first mismatch and retry that step.
Correct result without a reason	Ask for a trace and a changed case before marking independent understanding.
An unsupported wider claim	Ask which supplied evidence supports the claim and which further observation would be needed.

Offer an oral explanation, drawing, enlarged visual or fewer examples when that removes a reading/writing barrier. Keep the same reasoning goal.

Stretch secure learners by changing a **boundary** or assumption and asking them to defend the effect, not simply by assigning more of the same questions.

Assess and explain the answers

> Use the Debugging Strategies questions, teacher keys and criteria after the relevant attempts. Previously discussed cases are practice; protect the separately selected independent task.

Worked case answer

Teacher key · Worked case answer

Expected 2; buggy result 4. Put the addition inside the match condition. Test zero, one and two matches for 0,2,4.

Independent question

Independent question · withhold the key

A loop should run four times but runs five. You change its colour and stopping condition at once. Which single change is relevant to test first?

Expected independent answer

Teacher key · Expected independent answer

Test the stopping condition first while preserving the other code. Trace the counter at the **boundary**; changing colour provides no **evidence** about repetition.

What counts as evidence

Criterion	0 points	1 point	2 points
Result	Incorrect or absent	Partly correct	Correct with necessary conditions
Reasoning	No supporting trace	Some steps	Clear mechanism, calculation or source support
Limit	Overclaims certainty	Vague limitation	Relevant uncertainty or missing fact
Transfer	Copies without adapting	Adapts with prompts	Solves a changed case independently

Use 6/8 as a classroom checkpoint, with 2 required for both Result and Reasoning. This is a proposed teaching rule, not a validated certification threshold.

Do not count a coached answer as independent mastery; correct unsafe or unsupported claims regardless of total.

Feedback

Ask the learner to find the **evidence** that challenges this claim: “Any visible change helps diagnose the cause.” Keep the first attempt and an uncoached retry separate.

Original transfer answer

Teacher key · Original transfer answer

Expected 0,3,6; actual 3,6,9. Initialise score to 0, keeping the per-match addition. The zero-match case isolates the initialisation error. Rerun all three tests; changing only a displayed result would leave the cause intact.

Explanations for the additional workbook examples

W1 Write expectations

Workbook question · W1

Give expected results for remaining(6, 2), remaining(6, 6) and remaining(6, 0).

Teacher reasoning · W1

4, 0 and 6.

Teacher reasoning · W1

Apply the specified subtraction rule to each pair: 6 - 2 = 4, 6 - 6 = 0, and 6 - 0 = 6. Label them ordinary, full and unused cases. These expected answers come from the contract

before the program is run.

W2 Find a revealing case

Workbook question · W2

A faulty function always returns 4. Which of your three cases pass and fail?

Teacher reasoning · W2

Only (6, 2) passes. The full and unused cases fail.

Teacher reasoning · W2

Compare the faulty actual result 4 with each independently expected result. It matches the ordinary case expecting 4, but not the full case expecting 0 or unused case expecting 6.

One successful case therefore does not establish that the subtraction rule was implemented.

W3 Repair and retest

Workbook question · W3

A faulty function adds capacity and used. For (6, 2), state actual and expected. Repair it and name a **regression** case.

Teacher reasoning · W3

Actual 8, expected 4. Subtract used; rerun a prior case such as (6, 0), expecting 6.

W4 Test a new function independently

Workbook question · W4

Define stock(packs, loose) to return packs * 4 + loose. Create expected tests for (2, 3), (0, 3), (2, 0), then show which detects a faulty packs + loose body.

Teacher reasoning · W4

Expected 11, 3, 8. Faulty results 5, 3, 2; first and third reveal it. Use independent expectations in the test loop.

W5 Limit the conclusion

Workbook question · W5

All three stock tests pass. Does that prove correct handling of negative counts?

Teacher reasoning · W5

No. Negative counts are outside our nonnegative-**input** contract and were not tested; rejection behavior must first be defined.

Assess independent transfer

Independent question · withhold the key

Selected case: Define stock(packs, loose) to return packs * 4 + loose. Create expected tests for (2, 3), (0, 3), (2, 0), then show which detects a faulty packs + loose body.

Selected case answer

> **Teacher key · Selected case answer**
>
> Expected 11, 3, 8. Faulty results 5, 3, 2; first and third reveal it. Use independent expectations in the test loop.

Dimension	Evidence required
Explanation	Names the applicable idea or procedure and ties it to the supplied facts.
Trace and result	Preserves the given inputs and shows the relevant intermediate steps; distinguishes a prediction from an observed execution.
Limits	States the assumptions, missing evidence or conditions under which the result would change.

> **Teacher key · Selected case answer**
>
> Record each dimension as not yet, with support, or independent. Accept a valid alternative procedure with a defensible trace.
>
> Do not penalise an honestly recorded model failure if the learner correctly explains its **evidence** and limits.

Feedback that the learner uses

Name the earliest unsupported step and give one action to repair it. Have the learner make that repair now and explain why. Then assess the relevant skill on a fresh, fully specified case.

Confirm it was not previously practised with feedback; solve and check the new case before assigning it. A follow-up design prompt is a starting brief, not automatically an equivalent test.

Connect and extend learning

> Connect Debugging Strategies to Transparent Book Recommender, adapt support and plan a later changed-case check.

Transparent Book Recommender

When a ranking surprises you, inspect feature values, weights, arithmetic, and sorting separately.

Keep the decision, **input**, result, explanation and remaining limitation in the project **evidence** record. Retrieve this record next week rather than restarting the project.

Support and stretch

Trace one record or condition at a time with counters and labelled columns. Stretch: introduce a tie, **boundary** or missing field and declare its handling policy before rerunning.

Exit reflection

Ask for one decision the learner can now justify and one situation requiring more **evidence** or human review. Confidence ratings are separate from correct independent performance.

Bridge to the next week

Next: Week 7: Rows, Columns, and Data Types. Retrieve today's **evidence** and identify what carries forward and what is new.

Support and fresh reassessment

Provide a blank **input**s expected actual table. Extension: use assert for the same cases and explain why only the first failure is reported.

Independent W4 **evidence**: Expected 11, 3, 8. Faulty results 5, 3, 2; first and third reveal it. Use independent expectations in the test loop. Record whether the learner needed no help, a hint or explanation.

Fresh retry: bundles * 3 + singles with (3, 1), (0, 1), (3, 0) expects 10, 1, 9. A constant return 10 passes only the first.

Next week: Rows columns and schemas. Reuse this week's explicit contracts and **evidence**.

Plan the delayed return

Suggested return points in this level: ai-2/7, ai-2/9, ai-2/12. Use actual classroom dates, adjusting for holidays and gaps. One, three and six teaching weeks are scheduling choices, not a claimed optimal spacing.

At a delayed check, ask for the idea from memory and a changed application. Compare with the earlier independent response and record support used.

For a new level, revisit the previous capstone before checking the new prerequisites.

Record	Teacher evidence
Learner code and date	
Concept and case used; prior exposure	
Explanation / trace / limits	
Support and first repair	
Changed case and delayed outcome	
Next teaching action	

Use the **evidence** to choose reteaching or the next step. A completed page, confidence rating or attractive project does not by itself demonstrate understanding.

Project evidence from this case

Test the recommender's helper functions with normal, zero and **boundary input**s before **debug**ging the whole application.

Prepare, teach and assess

> Use this preparation summary alongside the detailed Debugging Strategies lesson; it is a

checklist rather than a replacement for the full sequence.

Teaching target

Teaching target: Design revealing tests for a function

Prepare the labelled worked-case picture, the matching workbook, fictional **input** cards and a place for individual responses. No paid AI account is required for the paper reasoning activities.

Session · Plan · Resources

Session: Session 1 · Understand · 45 min

Plan: 0–7: recall and prerequisite; 7–15: inspect the concrete case and predict; 15–30: teacher models the worked trace; 30–40: learners explain a step using the text and visual; 40–45: check the core idea.

Resources: Use Your mission, Notice the evidence, See how it works, Compare the cases, Words that unlock the idea and Follow a worked example; record the brief recall in the workbook.

Session: Session 2 · Practise · 45 min

Plan: 0–5: retrieve the procedure; 5–15: W2 completion practice; 15–23: explain and check with a partner; 23–30: each learner answers the misconception check; 30–40: targeted reteaching or a harder variation; 40–45: prepare the investigation.

Resources: Use Find and repair a mistake, Practise together, Run the investigation and Build your understanding; use the matching workbook W2 and misconception check.

Session: Session 3 · Investigate and transfer · 45 min

Plan: 0–5: retrieve the decision rule; 5–20: investigate or trace; 20–30: start the independent case; 30–38: connect the project; 38–43: act on feedback; 43–45: plan the return check.

Resources: Use Run the investigation, Connect your project, Show what you know, Study a complete case and Try a new case independently. Allow extra time for the full task set. Use a coding lab only when this week actually provides one; Your lesson route is the planning section.

Before class, solve the supported practice: A faulty function always returns 4. Which of your three cases pass and fail?

Supported practice answer

Reasoned practice answer: Only (6, 2) passes. The full and unused cases fail.

Check every learner before discussion: Use the worked case for Testing and **debug**ging functions. Proposed explanation: “A successful run proves the conclusion”. Decide whether this explanation is supported.

Point to the step or supplied fact that decides; explain your repair if needed.

Repair the misconception

Misconception repair: Passing this small suite supports these particular cases. It neither proves every allowed **input** correct nor checks invalid **input** handling.

A test outside the contract needs a separately specified expected behavior.

Support without giving away the answer

Support: read the exact case aloud, enlarge the labelled picture, supply **input** cards and model only the first deciding step. Accept a spoken explanation or drawing; retain the reasoning goal.

Extend the reasoning

Extension: Repair the body to bundle_count*6+loose. Require a **prediction** and a reason before comparing. Expected result: Expected results for (2,1),(0,3),(2,0) are 13,3,12.

Reason: Only bundles are groups of six; loose books are counted individually.

Protect the independent attempt

Independent assessment: withhold the reserved answer until the learner has attempted the new case. Record support separately from correctness.

Criterion	Not yet	With support	Independent
Explanation	Names an answer without a deciding fact	Uses a hint to link fact and decision	Explains the deciding fact in own words
Trace or test	Cannot connect input and result	Completes steps with prompts	Traces or tests a changed case accurately
Limits and revision	Claims more than evidence supports	Names a limit after a prompt	States a defensible limit and useful next test

Project checkpoint

Project checkpoint: Frame the need. Name the user, the problem, allowed **input**s and a safe simple starting method.

Use the weekly teacher answer for topic-specific reasoning. A saved progress tick is an activity record; judge mastery from the learner's explanation and **evidence**.

Use feedback, labs and repair paths

> Use feedback, earlier review and portfolios purposefully. Apply the coding routine only when this lesson has an executable lab.

Use the two formative questions as supported practice, not as independent mastery scores. Discuss the deciding **evidence** rather than accepting a guessed selection.

The feedback comes from the reviewed public worked case. Do not reveal the separately reserved independent teacher answer until the learner has attempted that case.

Ask students to predict before running code. Preserve errors and actual **output**s; collect a changed test and an explanation. The branch-**boundary** and indexing lessons deliberately include failure cases.

Use the earlier worked-explanation links to address a specific missing building block, then return to the current question. Adapt this suggestion after observing the learner; it is not an automatic diagnosis.

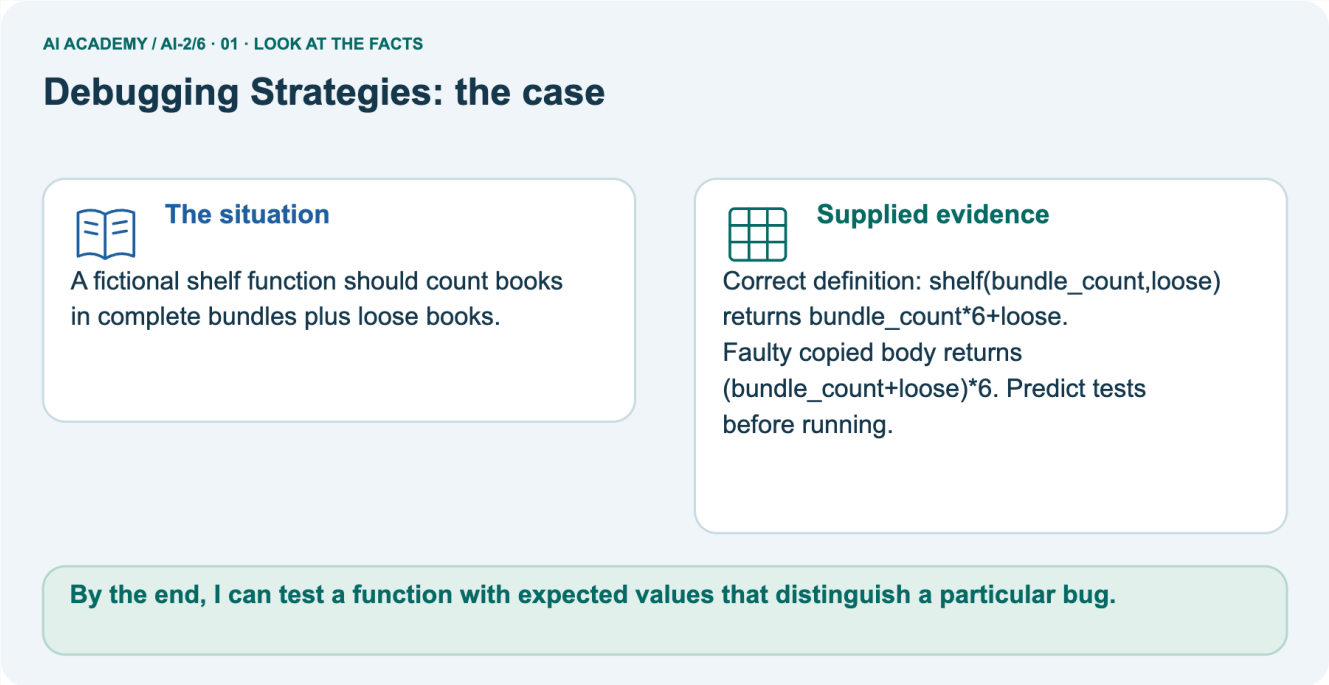
Collect the completed-work portfolio as **evidence**. Record support separately from correctness, and assess explanation, trace/test and limitations. Progress ticks and quiz selections alone do not establish mastery.

Visual concept journey

> Show one Debugging Strategies graphic at a time, request a prediction and discuss its trace before the independent assessment.

Use one picture at a time. Hide a worked result before asking for a **prediction**, then reveal it for feedback. These are supported teaching cases; retain the separate independent assessment for a fresh first attempt.

01. Debugging Strategies: the case



Follow the labelled picture with the matching explanation. Open the vector diagram

A fictional shelf function should count books in complete bundles plus loose books. Supplied facts: Correct definition: shelf(bundle_count,loose) returns bundle_count*6+loose. Faulty copied body returns (bundle_count+loose)*6.

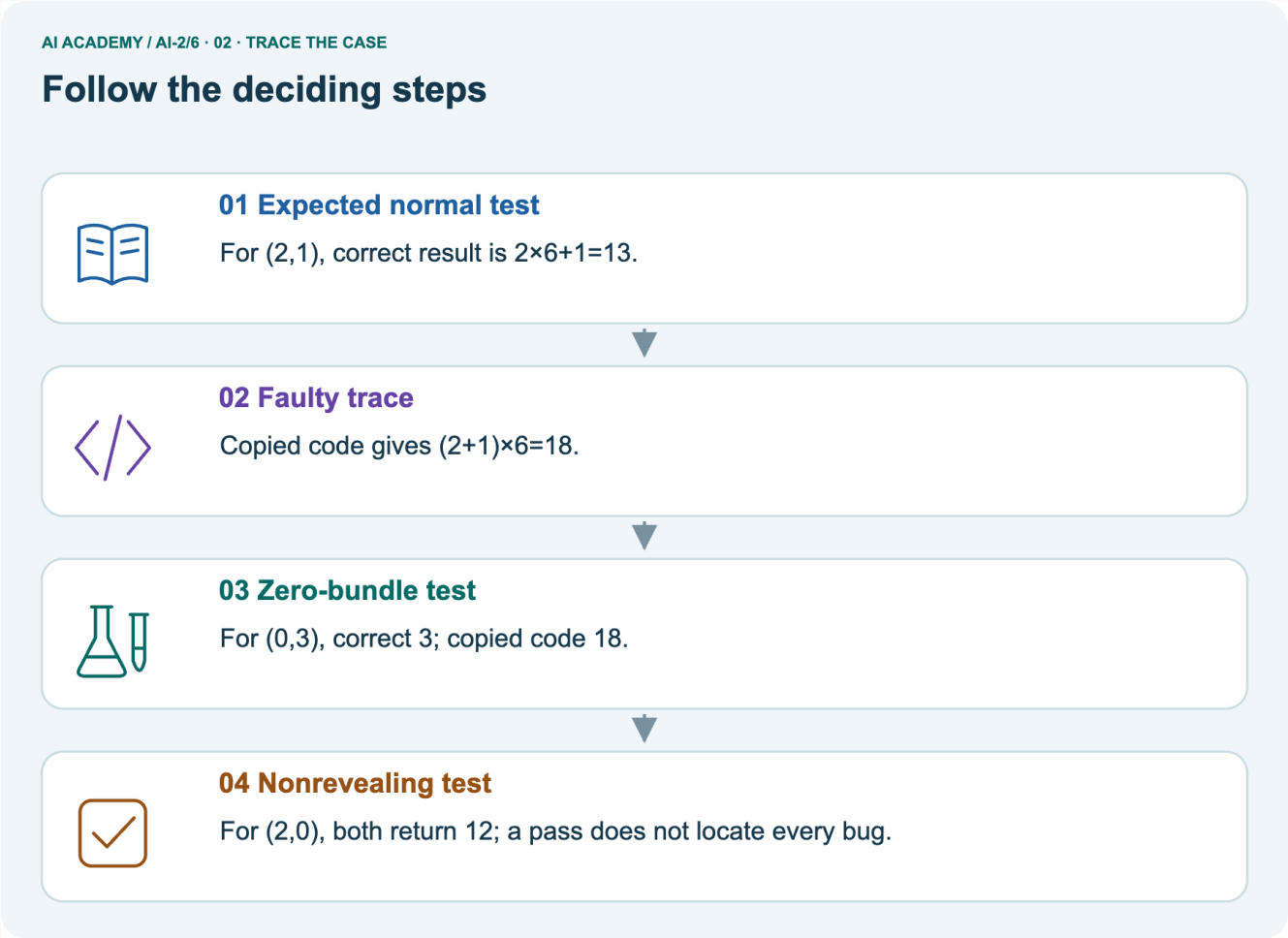
Predict tests before running.

> Point to the given fact that matters most. Explain why it matters.

Discuss while showing the picture

Teacher prompt: Point to the given fact that matters most. Explain why it matters. Ask learners to describe one label or arrow in ordinary words. Use the adjacent transcript if the image is difficult to see.

02. Follow the deciding steps



Follow the labelled picture with the matching explanation. Open the vector diagram

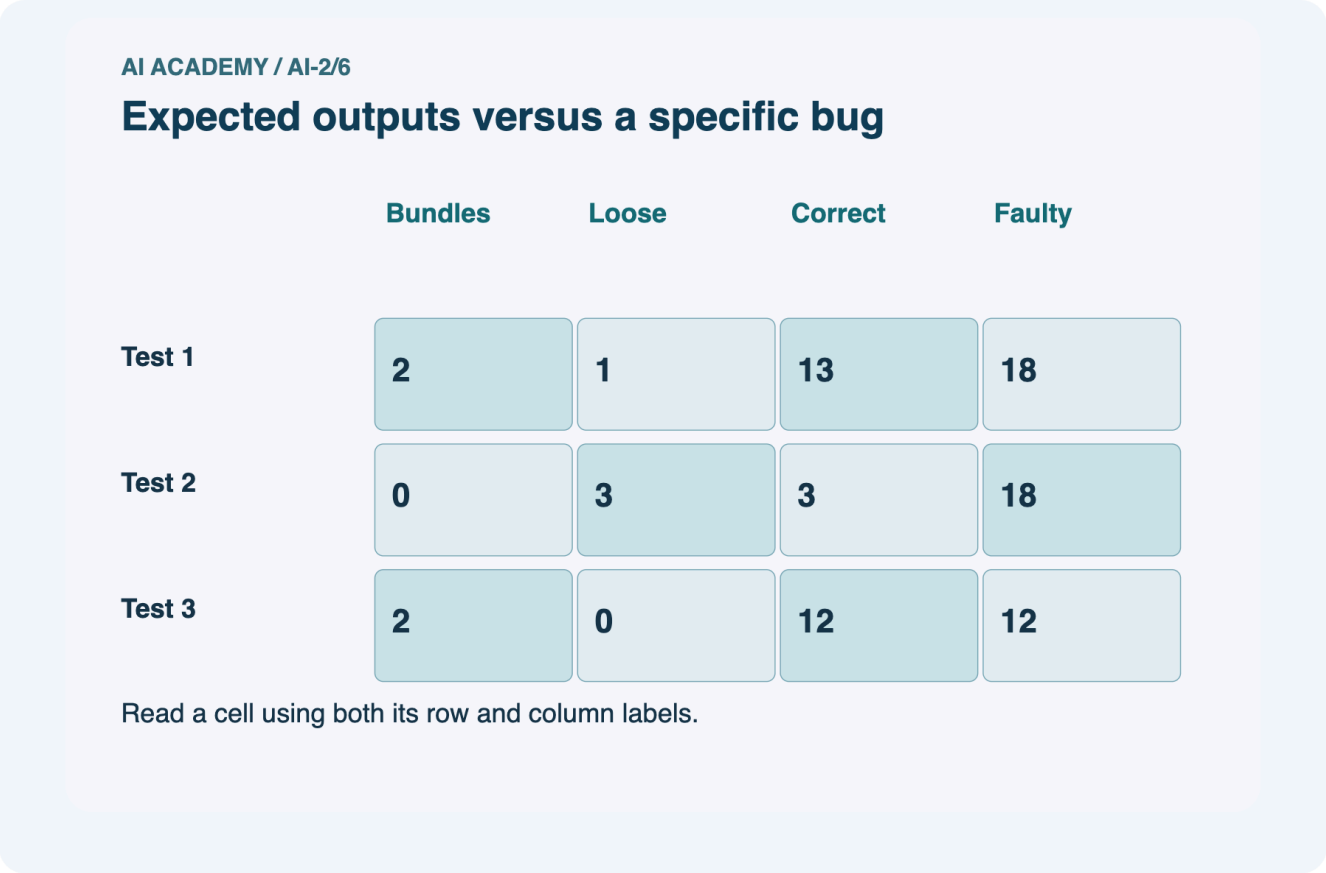
Expected normal test: For (2,1), correct result is 2×6+1=13. → Faulty trace: Copied code gives (2+1)×6=18. → Zero-bundle test: For (0,3), correct 3; copied code 18. → Nonrevealing test: For (2,0), both return 12; a pass does not locate every bug.

> Point to each numbered stage and explain which recorded **input** it uses.

Discuss while showing the picture

Teacher prompt: Point to each numbered stage and explain which recorded **input** it uses. Ask learners to describe one label or arrow in ordinary words. Use the adjacent transcript if the image is difficult to see.

03. Expected outputs versus a specific bug



Follow the labelled picture with the matching explanation. Open the vector diagram

Expected **output**s versus a specific bug. Test 1: Bundles = 2, Loose = 1, Correct = 13, Faulty = 18. Test 2: Bundles = 0, Loose = 3, Correct = 3, Faulty = 18. Test 3: Bundles = 2, Loose = 0, Correct = 12, Faulty = 12.

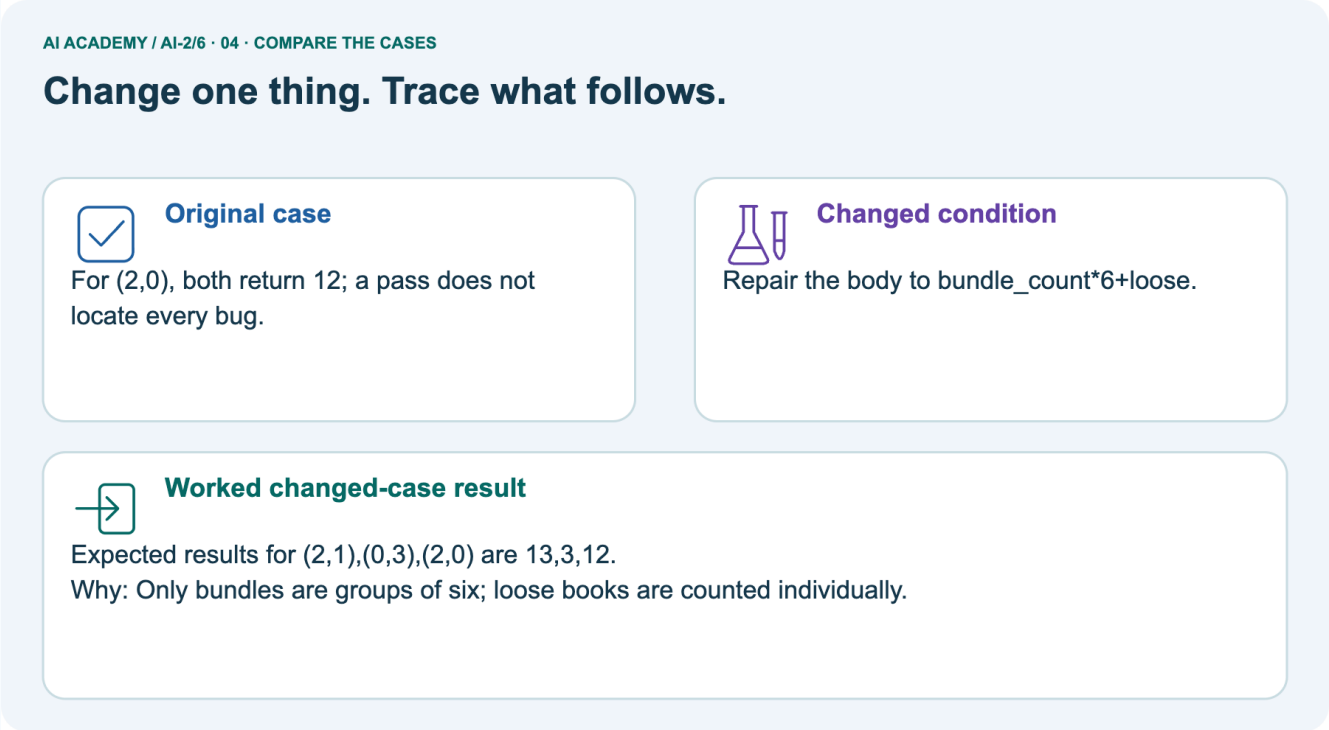
Read the labels and values. Explain the relationship the model shows.

Discuss while showing the picture

Teacher prompt: Read the labels and values. Explain the relationship the model shows. Ask learners to describe one label or arrow in ordinary words. Use the adjacent transcript if the image is difficult to see.

Check row/column meanings, units and denominators before discussing a difference. Supplied toy values are not real class or model results.

04. Change one thing. Trace what follows.



Follow the labelled picture with the matching explanation. Open the vector diagram

Original case: For (2,0), both return 12; a pass does not locate every bug. Changed condition: Repair the body to bundle_count*6+loose. Expected result: Expected results for (2,1),(0,3),(2,0) are 13,3,12.

Reason: Only bundles are groups of six; loose books are counted individually.

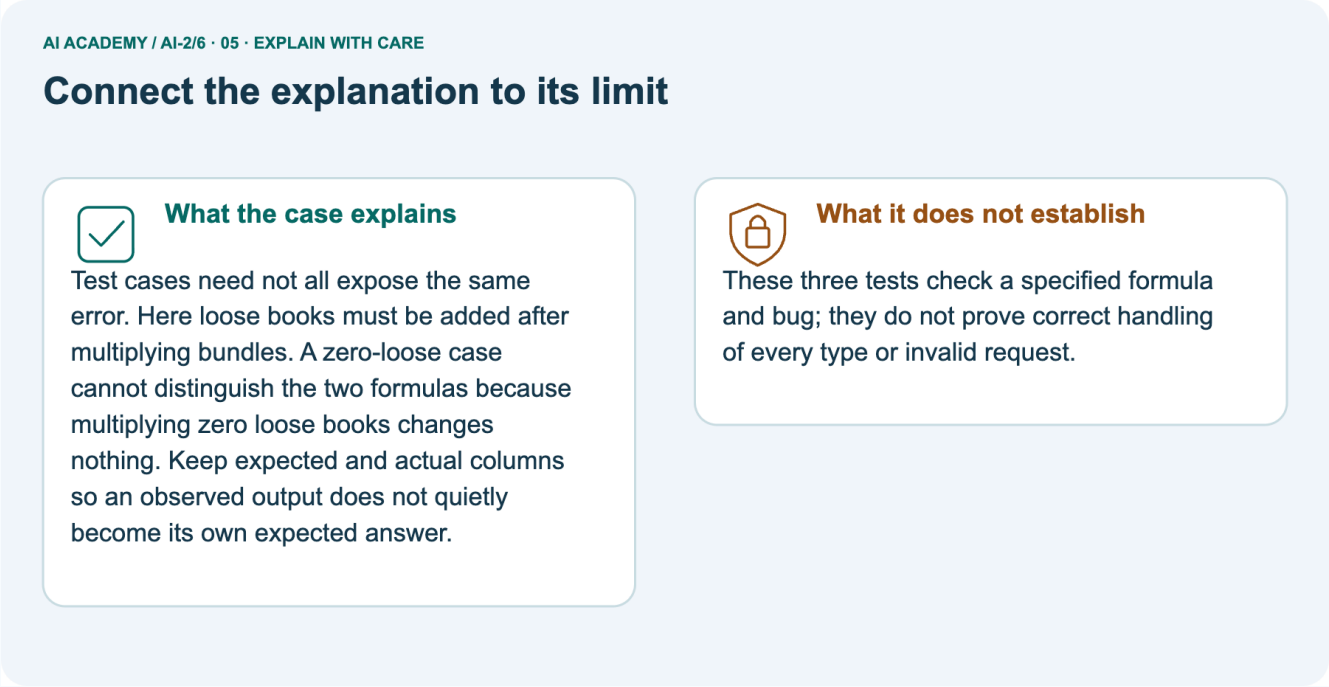
Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude.

Discuss while showing the picture

Teacher prompt: Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude. Ask learners to describe one label or arrow in ordinary words.

Use the adjacent transcript if the image is difficult to see. Changed-case reasoning: Only bundles are groups of six; loose books are counted individually.

05. Connect the explanation to its limit



Follow the labelled picture with the matching explanation. Open the vector diagram

Test cases need not all expose the same error. Here loose books must be added after multiplying bundles. A zero-loose case cannot distinguish the two formulas because multiplying zero loose books changes nothing.

Keep expected and actual columns so an observed **output** does not quietly become its own expected answer.

Limit: These three tests check a specified formula and bug; they do not prove correct handling of every type or invalid request.

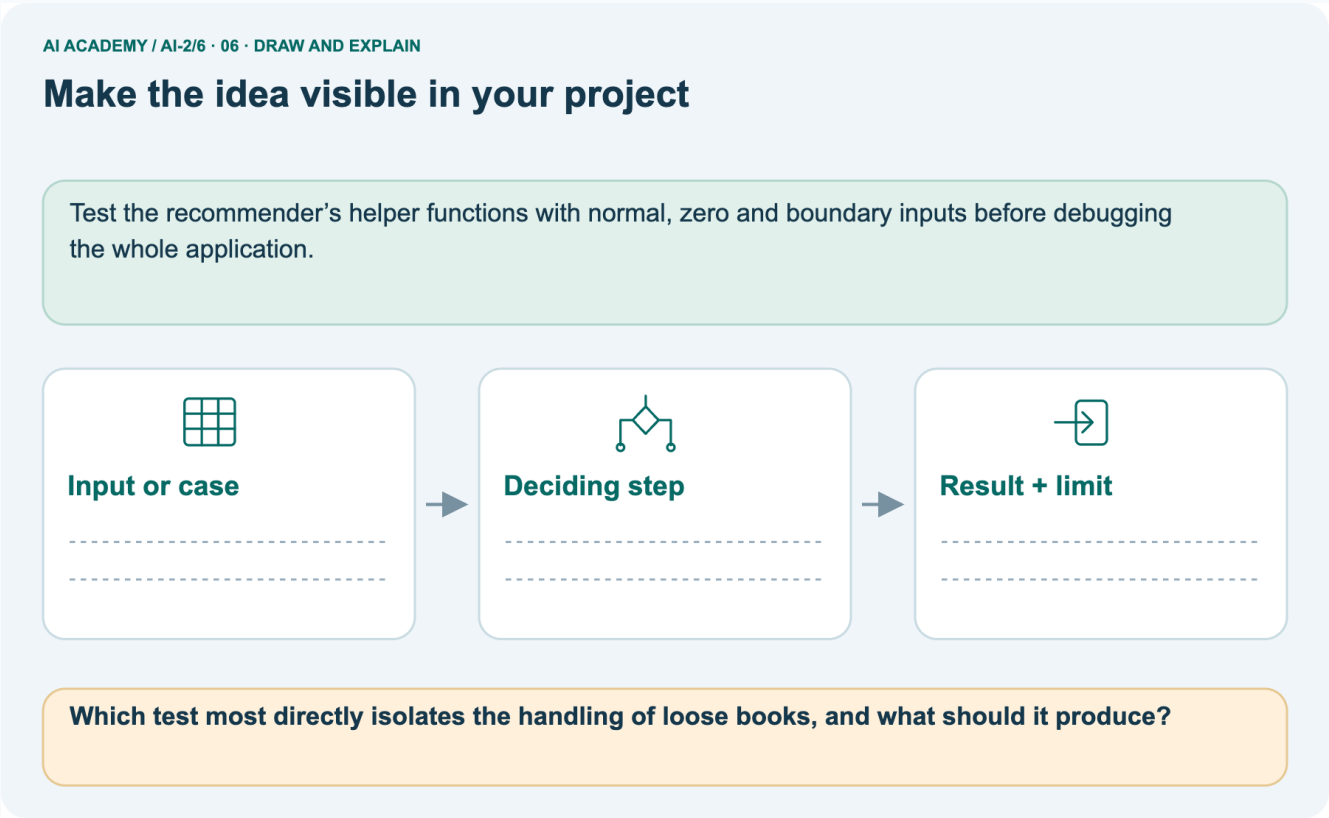
Explain the case, then point to a limit on the claim.

Discuss while showing the picture

Teacher prompt: Explain the case, then point to a limit on the claim. Ask learners to describe one label or arrow in ordinary words. Use the adjacent transcript if the image is difficult to see.

Keep the stated scope: These three tests check a specified formula and bug; they do not prove correct handling of every type or invalid request.

06. Make the idea visible in your project



Follow the labelled picture with the matching explanation. Open the vector diagram

Test the recommender's helper functions with normal, zero and **boundary input**s before **debug**ging the whole application. Draw your **input**, deciding step, result and limit.

Supported question: Which test most directly isolates the handling of loose books, and what should it produce?

> Draw your project trace and justify the deciding step using **evidence**.

Discuss while showing the picture

Teacher prompt: Draw your project trace and justify the deciding step using **evidence**. Ask learners to describe one label or arrow in ordinary words. Use the adjacent transcript if the image is difficult to see.

> Supported practice key: (0,3) isolates loose books: expected 3. The faulty body gives 18.

Save or print your responses for your portfolio. Progress and drafts on the hosted website stay in the current browser. Use only fictional or teacher-approved data.